

Article

PP-EDUVec: Privacy-Preserving Intelligent Management Algorithms for Educational-Corpus Vector Databases Under Retrieval-Augmented Learning

Shiming Fu ^{1,*}, Fen Liu ², Jie Zhou ³, Jianping Cai ^{4,*}  and Zijie Pan ⁴¹ School of Artificial Intelligence, Chongqing University of Education, Chongqing 400067, China² Chongqing Research Institute of China Coal Technology & Engineering Group, Chongqing 400039, China³ Faculty of Computer Science and Engineering, School of Innovation Engineering, Macau University of Science and Technology, Macau 999078, China⁴ Faculty of Data Science, City University of Macau, Macau 999078, China

* Correspondence: fushm@cque.edu.cn (S.F.); jpcai@cityu.edu.mo (J.C.)

Abstract

Educational platforms increasingly rely on vector databases to store and retrieve embedding representations of large-scale learning corpora (e.g., lecture notes, assignments, feedback, and student Q&A) for retrieval-augmented generation and analytics. However, directly indexing educational text embeddings raises privacy risks (student identities, sensitive performance signals, and protected attributes) and creates a management challenge: embeddings drift as curricula evolve, access policies change, and new content arrives continuously. This paper studies privacy-preserving intelligent management of educational-corpus vector libraries and proposes a novel, end-to-end algorithmic framework that jointly optimizes (i) privacy leakage control, (ii) retrieval quality, and (iii) operational efficiency under streaming updates. We introduce a hierarchical policy-aware vector lifecycle model, a privacy budget scheduler for adaptive re-embedding and re-indexing, and a secure-aware clustering-and-routing mechanism that supports fast query-time filtering with minimal accuracy loss. The resulting system, PP-EDUVec, enables compliant similarity search across multi-tenant educational data while automatically maintaining index health (freshness, redundancy, and utility) over time. On the EDU-Mix benchmark, PP-EDUVec achieves Recall@10 = 0.835 while reducing representation leakage (LeakRep) from 0.215 to 0.136 (−36.7%) and access-pattern leakage (LeakAP) from 0.398 to 0.255 (−35.9%), and lowering mean latency from 42.1 ms to 33.4 ms (−20.7%) and weekly maintenance time from 55.0 to 35.8 min/week (−34.9%) compared with PostFilter.

Keywords: privacy preservation; vector databases; policy-aware retrieval; streaming maintenance



Academic Editor: Arkaitz Zubiaga

Received: 9 January 2026

Revised: 11 February 2026

Accepted: 11 February 2026

Published: 25 February 2026

Copyright: © 2026 by the authors.

Licensee MDPI, Basel, Switzerland.

This article is an open access article distributed under the terms and conditions of the [Creative Commons Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.

1. Introduction

Educational platforms are rapidly adopting retrieval-augmented language technologies to support tutoring, question answering, feedback generation, and content navigation. A common infrastructure choice is a vector database: educational documents are chunked, embedded into vectors, and indexed for similarity search so that a query embedding can retrieve semantically related materials. While effective, this workflow introduces a distinct challenge in education: the underlying corpora often contain highly sensitive information (student identities, performance feedback, protected attributes, or disciplinary records),

and access must be strictly controlled by role, course, and time. In addition, educational content is inherently dynamic—courses evolve weekly, assignments are revised, solutions become available after deadlines, and data may need to be deleted due to retention or compliance requirements. This combination yields a management problem that is not addressed by standard vector database designs: **how to maintain an educational-corpus vector library over time such that retrieval remains accurate and efficient, policies are enforced by design, and privacy leakage is minimized under continual updates.**

Existing work related to vector databases and RAG typically treats (i) access control, (ii) privacy protection, and (iii) streaming index maintenance as largely separable problems. Educational deployments couple them tightly: role/time policies change admissible regions over time, curricula drift triggers repeated (re-)releases of representations, and query execution can leak sensitive information via access traces. PP-EDUVec targets this joint setting by unifying policy-first physical index structuring, access-trace mitigation, and budgeted lifecycle scheduling in a single online management loop.

We study this setting as (privacy-preserving intelligent management of educational-corpus vector libraries). Formally, at time t the system stores a library $\tilde{V}_t = \{(\tilde{x}_i, \mu_i)\}$ and an index $\mathcal{I}_t$ that supports policy-compliant retrieval $\text{Retrieve}(q, a, k)$ as defined in Equation (6). The library evolves through an update stream Δ_t (insertions, deletions, meta-data changes, and re-embeddings), and privacy must be tracked cumulatively via per-unit budgets $\mathcal{E}_t(p) \leq \varepsilon_{\max}(p)$ in Equation (9). The challenge is multi-objective: maximizing retrieval utility (e.g., Recall@10 and nDCG@10) while minimizing operational costs (latency, memory, maintenance time) and two privacy-risk channels that arise naturally in this context: (A) representation leakage from stored embeddings and (B) access-pattern leakage from deterministic index probes and repeated retrieval behaviors. Even when some educational materials are publicly accessible (e.g., open course notes), privacy risks persist because the vectorized form and the usage traces can expose information not explicit in raw documents: embeddings can enable membership-style inference about restricted subsets and repeated releases accumulate exposure, while access traces can reveal sensitive user intent and behavior (and in educational settings, may correlate with performance-related signals). Our goal is, therefore, to manage the retrieval substrate so that policy compliance is enforced by design, representation releases are privacy-accounted, and access-trace leakage is explicitly reduced and measured.

Existing practices typically treat these aspects in isolation. Many systems apply access control after candidate retrieval (post-filtering), which can waste computation and can create side-channel risks if unauthorized partitions are accessed during the search process. Privacy protection is often applied as a one-time sanitization step at insertion, ignoring the fact that educational corpora undergo repeated updates and re-indexing, which cumulatively consume privacy budgets and may require careful scheduling. Finally, drift induced by curriculum evolution degrades retrieval relevance unless embeddings and index structures are refreshed, but aggressive refreshing can exceed privacy budgets or impose unacceptable maintenance overhead. These tensions suggest the need for a unified management algorithm that couples policy structure, privacy accounting, and lifecycle decisions.

We position PP-EDUVec relative to recent studies of privacy risks in retrieval-augmented generation, which demonstrate that retrieval pipelines introduce distinct leakage surfaces beyond the base LLM [1]. PP-EDUVec is complementary: rather than modifying the generator, we focus on the vector-database management layer that executes retrieval under strict role/time authorization and continual updates. We further relate to privacy-preserving vector search engines in federated/TEE settings [2], which adopt different trust assumptions and target different deployment regimes.

To address this need, we propose **PP-EDUVec**, a novel framework for managing educational vector libraries under continual updates. PP-EDUVec integrates three new algorithmic components. First, it introduces **policy-first index structuring** via compact policy signatures, enabling candidate generation that is aligned with authorization constraints before expensive similarity search. Second, it adds **secure-aware cluster routing** that injects controlled stochasticity into index probing to reduce access-pattern distinguishability without directly perturbing ranking scores. Third, it presents **BARES**, a budgeted adaptive lifecycle scheduler that decides when to re-embed, re-release, or compact vectors based on predicted utility gains, maintenance costs, and remaining privacy budgets, maintaining long-horizon compliance under continual observation.

Our contributions are summarized as follows:

1. We formalize the problem of privacy-preserving intelligent management for educational-corpus vector libraries as a streaming, policy-constrained optimization with explicit privacy-budget accounting over time (Section 3).
2. We propose PP-EDUVec, combining policy-first structuring, secure-aware routing, and budgeted lifecycle scheduling, yielding a practical end-to-end management algorithm for $(\tilde{\mathcal{V}}_t, \mathcal{I}_t)$ under Δ_t (Section 4).
3. We demonstrate through comprehensive experiments that PP-EDUVec achieves a favorable privacy–utility frontier, significantly reducing representation and access-pattern leakage proxies while maintaining high retrieval quality and lowering maintenance cost compared with strong baselines (Section 5).

The remainder of the paper is organized as follows. Section 2 reviews related work and clarifies the scope of PP-EDUVec relative to secure search and privacy analyses of RAG. Section 3 formalizes the setting, threat model, and privacy accounting. Section 4 presents PP-EDUVec, including policy-first indexing, secure-aware routing, and the BARES lifecycle scheduler. Section 5 evaluates privacy, retrieval utility, and operational cost under streaming educational workloads. Section 6 discusses limitations and concludes.

2. Related Work

Vector libraries for educational corpora sit at the intersection of similarity search infrastructure, retrieval-augmented generation, and privacy-preserving data management. PP-EDUVec draws from these lines while addressing a gap: continuous, policy-aware, privacy-budgeted lifecycle management for embedding collections $\tilde{\mathcal{V}}_t$ under dynamic updates Δ_t .

2.1. Vector Similarity Search and Dynamic Indexing

Approximate nearest neighbor (ANN) search is the computational backbone of modern vector databases. Classic quantization-based approaches, including product quantization, enable memory-efficient search at scale [3]. Graph-based methods such as hierarchical navigable small world graphs (HNSW) provide strong empirical accuracy–latency trade-offs [4]. System implementations like FAISS made large-scale ANN practical via a family of indexing strategies and GPU acceleration [5], while DiskANN demonstrated billion-scale, SSD-friendly single-node retrieval [6]. ScaNN further improved efficiency via anisotropic vector quantization [7].

Recent years have seen vector database management mature into a systems topic with dedicated surveys and tutorials [8–10], emphasizing ingestion, indexing, filtering, and multi-tenant operations. At the same time, research has begun to target dynamic and streaming vector indexing: incremental maintenance of IVF-style indexes, dynamic range-filtering for ANN [11], and persistent real-time vector DB designs tailored for LLM applications [12]. In parallel, privacy-preserving and federated vector search has emerged

as an adjacent line, e.g., FedVSE [2]. These works primarily optimize search speed, freshness, and storage under large-scale workloads. PP-EDUVec focuses on the additional constraints of policy-first retrieval and privacy-aware lifecycle for continuously changing educational corpora, where management overhead and privacy budget consumption are first-class constraints.

2.2. Retrieval-Augmented Language Models and Educational Retrieval

Retrieval-augmented paradigms have become central to question answering and long-context reasoning. Dense retrieval via bi-encoders such as DPR [13] and pretraining-time retrieval such as REALM [14] established strong baselines for open-domain QA. Generative retrieval-augmented models that combine retrieval with conditional generation, such as RAG [15], and architectures that fuse many retrieved passages (FiD) [16], demonstrate consistent gains in factuality and coverage. Large-scale retrieval-augmented LMs, including RETRO, highlight the benefits of retrieving from massive external corpora. Recent surveys systematize RAG design choices and evaluation, reflecting the rapid expansion of the area [17].

In educational platforms, similar retrieval patterns arise for tutoring, feedback generation, and content recommendation; however, the retrieved material is often sensitive and access-controlled. Moreover, new attack surfaces appear when retrieval is integrated with generation: recent work studies membership inference against retrieval-augmented pipelines. PP-EDUVec complements RAG-style systems by maintaining $\tilde{\mathcal{V}}_i$ so that policy-compliant retrieval in Equation (6) remains accurate and efficient under updates while controlling privacy risks.

2.3. Differential Privacy and Leakage from Learned Representations

Differential privacy (DP) provides a principled notion of limiting information leakage from data releases [18,19]. In machine learning, DP-SGD and related techniques enable training models with formal privacy guarantees [20], while alternative frameworks such as PATE provide privacy via teacher ensembles. Privacy accounting tools, including Rényi DP, refine composition analysis over many iterations [21]. More recently, DP has been extended to fine-tuning and user-level protection regimes for language models, which is relevant when educational systems personalize or adapt models over time [22].

Beyond training-time privacy, a growing literature shows that representations and model outputs can leak memorized or identifying information. Membership inference demonstrates that adversaries can infer whether a record was in a training set [23], memorization studies reveal that models can retain rare sequences [24], and large language models may enable extraction of training examples [25]. These findings motivate PP-EDUVec's emphasis on release-time sanitization $\tilde{x} = \mathcal{M}(x; \epsilon, \delta)$ (Equation (8)) and budgeted re-release over time (Equation (9)), since educational vector libraries may undergo repeated updates and re-embeddings that accumulate privacy loss.

2.4. Access Control, Multi-Tenancy, and Secure Search

Educational systems are inherently multi-tenant and multi-role, motivating access-control models such as RBAC [26] and attribute-based access control (ABAC) guidance [27]. Recent NIST guidance also highlights access control in the presence of attribute encryption, which is relevant to enforcing policy constraints in encrypted or partially trusted environments [28]. A central challenge is enforcing $\pi(a, \mu)$ efficiently without leaking unauthorized information through side channels.

Searchable symmetric encryption (SSE) provides cryptographic methods for keyword search over encrypted data [29] and scalable constructions supporting richer queries [30]. Recent work further systematizes SSE design and deployment considerations [31] and

strengthens system-wide security properties [32]. For similarity search, secure kNN-style protocols have been explored to compute nearest neighbors without exposing raw vectors [33], though such approaches often incur substantial computational overhead at scale. PP-EDUVec is complementary: it targets the practical regime of high-throughput vector search by combining policy-first structuring (signature-partitioned indexing) with secure-aware routing to reduce access-pattern distinguishability, rather than relying solely on heavy cryptographic primitives.

2.5. Streaming Data, Drift, and Lifecycle Management

Vector libraries over educational corpora are subject to curriculum drift, frequent edits, and retention-driven deletions. Concept drift has been extensively studied in streaming learning systems [34], with recent surveys focusing on modern monitoring and detection pipelines [35]. Streaming clustering algorithms such as STREAM provide foundations for incremental structure maintenance [36]. Privacy under continual releases has also been studied in the “continual observation” model, where repeated publication over time must be carefully composed [37]. PP-EDUVec builds on these perspectives by explicitly modeling an update stream Δ_t and introducing a budgeted adaptive lifecycle scheduler (BARES) that selects re-embedding and re-release actions according to predicted utility gains and remaining privacy budgets.

2.6. Educational Data Mining and Privacy in Learning Analytics

Educational data mining and learning analytics have highlighted the value of mining student interactions and instructional content [38,39]. At the same time, the field has emphasized ethical and privacy challenges due to the sensitivity of learner data [40,41]. Recent work continues to document practitioner readiness and governance challenges in learning analytics deployments [42], and empirical studies examine how students perceive privacy trade-offs when learning analytics are used in higher education [43]. While prior work articulates privacy principles and risks in educational contexts, relatively little work addresses vector-database-specific risks such as embedding leakage and access-pattern inference under retrieval-augmented applications. PP-EDUVec contributes by focusing on this emerging operational layer: how to maintain an educational-corpus vector library $\tilde{\mathcal{V}}_t$ that is simultaneously accurate, efficient, policy-compliant, and privacy-budgeted over time.

Secure search/SSE and secure similarity search protocols provide strong cryptographic protections but typically incur substantial overhead at the scale and latency targets of production vector databases. Federated/TEE-based vector search engines provide an alternative trust model and can enforce privacy across organizations, but they address different deployment assumptions than our single-operator educational setting with strict role/time policies and continual updates. Finally, recent work studies privacy risks in RAG at the LLM/prompting layer (including explicit extraction attacks on retrieval corpora), motivating stronger guarantees and mitigations in the retrieval substrate itself. PP-EDUVec complements these directions by focusing on the streaming vector-database management layer and by jointly optimizing policy compliance, privacy-budgeted releases, and access-trace mitigation.

3. Preliminaries

3.1. Data, Vectors, and Retrieval

We model an educational corpus evolving over discrete time $t \in \{1, 2, \dots\}$. Let $\mathcal{D}_t$ be the document set at time t . Each document $d \in \mathcal{D}_t$ is split into chunks by a deterministic chunker $\text{Chunk}(\cdot)$:

$$\mathcal{C}(d) = \{c_{d,1}, \dots, c_{d,n(d)}\}. \quad (1)$$

A chunk c is embedded by an encoder $f_\theta : \mathcal{X} \rightarrow \mathbb{R}^m$ to obtain $x = f_\theta(c) \in \mathbb{R}^m$. Each embedding is paired with metadata μ (tenant/course identifiers, timestamps, author role, sensitivity, etc.), and we denote an entry using $z = (x, \mu)$. The (unsanitized) vector library at time t is

$$\mathcal{V}_t = \{z_i\}_{i=1}^{N_t}. \quad (2)$$

An index $\mathcal{I}_t$ supports nearest-neighbor candidate generation for a query embedding $y \in \mathbb{R}^m$:

$$\text{Cand}_{\mathcal{I}_t}(y, k') \subseteq \mathcal{V}_t. \quad (3)$$

Given a similarity function $s(\cdot, \cdot)$ (cosine or inner product), the top- k operator over an eligible set $\mathcal{S}$ is

$$\text{TopK}(y, k; \mathcal{S}) = \arg \max_{S \subseteq \mathcal{S}, |S|=k} \sum_{(x, \mu) \in S} s(y, x). \quad (4)$$

3.2. Policies, Stream Updates, and Management Problem

Educational data is multi-tenant and multi-role. Let $\mathcal{U}$ be users and $\mathcal{R}$ roles; a query is issued by a principal $a = (u, r) \in \mathcal{U} \times \mathcal{R}$. Each entry is governed by an access policy predicate

$$\pi(a, \mu) \in \{0, 1\}, \quad (5)$$

where $\pi(a, \mu) = 1$ means authorized. For a query text q , let $y = f_\theta(q)$. Policy-compliant retrieval is defined as

$$\text{Retrieve}(q, a, k) = \text{TopK}(y, k; \text{Cand}_{\mathcal{I}_t}(y, k') \cap \mathcal{V}_t^{(a)}), \quad \mathcal{V}_t^{(a)} = \{(x, \mu) \in \mathcal{V}_t : \pi(a, \mu) = 1\}, \quad (6)$$

with internal depth $k' \geq k$. The corpus evolves via an update stream $\{\Delta_t\}$, where each Δ_t is a set of operations:

$$\Delta_t \subseteq \{\text{Insert}(z), \text{Delete}(id), \text{Update}(id, \mu'), \text{ReEmbed}(id, \theta')\}. \quad (7)$$

A management algorithm maps $(\mathcal{V}_{t-1}, \mathcal{I}_{t-1}, \Delta_t)$ to $(\mathcal{V}_t, \mathcal{I}_t)$, aiming to maintain high retrieval quality and low operational cost under continuous changes while enforcing $\pi(a, \mu)$.

3.3. Threat Model, Privacy Accounting, and Metrics

We assume an adversary can observe (i) stored vectors (or a subset), (ii) query-time access patterns and returned results, and (iii) public background knowledge. The goal is to infer sensitive information about individuals (e.g., membership, authorship, grades, protected attributes). Each entry has a sensitivity label $\sigma(\mu) \in \{0, 1, \dots, S\}$. We model vector release via a mechanism

$$\tilde{x} = \mathcal{M}(x; \varepsilon, \delta), \quad (8)$$

and denote the sanitized library by $\tilde{\mathcal{V}}_t = \{(\tilde{x}_i, \mu_i)\}_{i=1}^{N_t}$. Because updates and refreshes can trigger repeated releases, we track privacy spending per protected unit. Let $g(i)$ map entry i to a protected unit p (e.g., student id or document id). The cumulative budget up to time t is

$$\mathcal{E}_t(p) = \sum_{\tau \leq t} \varepsilon_\tau(p), \quad \mathcal{E}_t(p) \leq \varepsilon_{\max}(p) \quad \forall p, t. \quad (9)$$

What “privacy” means here: unless stated otherwise, we use formal (ε, δ) -differential privacy to refer to the stored-vector release channel: the mechanism $\mathcal{M}(\cdot; \varepsilon, \delta)$ in (8) is

instantiated as a standard (ϵ, δ) -DP release mechanism for ℓ_2 -bounded vectors (after clipping), and repeated (re-)releases compose over time. Equation (9), therefore, acts as a privacy accountant that tracks cumulative spending per protected unit p and enforces $\mathcal{E}_t(p) \leq \epsilon_{\max}(p)$ as a hard system constraint. **DP scope vs. access traces.** Our DP guarantee applies to stored sanitized vectors. We do not claim DP for query-time access traces; instead, we mitigate trace distinguishability via secure routing (Section 4.2.4) and evaluate it via an explicit trace-inference attacker (LeakAP) in Section 5.1.3.

We focus on two leakage channels: representation leakage from stored vectors and access-pattern leakage at query time, reported via proxies LeakRep and LeakAP (lower is better).

We evaluate management algorithms by retrieval utility (Recall@ k , nDCG@ k), operational efficiency (average query latency $\overline{\text{Lat}}$, average maintenance cost $\overline{\text{Maint}}$, and memory), and curriculum adaptivity via freshness (fraction of queries for which a relevant recent item within window W appears in top- k when such items exist). The objective is to maximize utility and freshness while minimizing operational cost and leakage, subject to strict policy compliance in (6) and privacy constraints in (9).

4. Methodology

This section presents **PP-EDUVec**, a privacy-preserving intelligent management framework for educational-corpus vector libraries. PP-EDUVec maintains a perturbed library $\tilde{\mathcal{V}}_t = \{(\tilde{x}_i, \mu_i)\}$ and an index $\mathcal{I}_t$ under an update stream Δ_t , enforcing policy-compliant retrieval in (6) and budget constraints in (9). The core novelty is a joint design of (i) policy-aware index structuring, (ii) secure routing that reduces access-pattern leakage, and (iii) a budgeted lifecycle scheduler that decides when and what to (re-)release.

4.1. System Overview

PP-EDUVec manages an evolving educational-corpus vector library by coupling **data sanitization**, **policy-aware organization**, and **budgeted maintenance** into a single online loop. At time t , the system state consists of (i) a sanitized vector library $\tilde{\mathcal{V}}_t = \{(\tilde{x}_i, \mu_i)\}_{i=1}^{N_t}$ and (ii) an index collection $\mathcal{I}_t$ used to support policy-compliant retrieval in (6). The state transitions are driven by a stream of updates Δ_t (insert, delete, metadata update, and re-embedding operations) while respecting cumulative privacy constraints (9). Formally, PP-EDUVec applies a management operator

$$(\tilde{\mathcal{V}}_{t-1}, \mathcal{I}_{t-1}, \Delta_t) \xrightarrow{\text{Manage}} (\tilde{\mathcal{V}}_t, \mathcal{I}_t), \quad (10)$$

whose design objective is to maximize retrieval utility and freshness subject to strict policy correctness and bounded privacy loss, while keeping query latency and maintenance overhead practical. Algorithm 1 summarizes PP-EDUVec's end-to-end workflow and how its components interact during ingestion, querying, and weekly maintenance.

Ingestion and sanitization. For each newly inserted or refreshed chunk c_i , PP-EDUVec first computes a raw embedding $x_i = f_\theta(c_i) \in \mathbb{R}^m$ and then applies a two-step sanitization interface (norm clipping followed by a privacy mechanism) before the vector enters the persistent library:

$$x_i = f_\theta(c_i), \quad \bar{x}_i = \text{clip}(x_i; B) = x_i \cdot \min\{1, B/\|x_i\|_2\}, \quad \tilde{x}_i = \mathcal{M}(\bar{x}_i; \epsilon_i, \delta_i). \quad (11)$$

Here, $B > 0$ bounds the ℓ_2 sensitivity of released vectors, and $\mathcal{M}$ is the release mechanism introduced in (8). In our evaluation, $\mathcal{M}$ is instantiated as the Gaussian mechanism applied to clipped vectors: $\tilde{x} = \bar{x} + \mathcal{N}(0, \sigma^2 I_m)$ with $\sigma = \frac{B\sqrt{2\ln(1.25/\delta)}}{\epsilon}$. This makes the (ϵ, δ) interpretation operational and reproducible; the framework is compatible with other (ϵ, δ) -DP release

mechanisms for bounded vectors. The privacy parameters (ϵ_i, δ_i) are not fixed constants: PP-EDUVec allocates them adaptively based on the sensitivity level $\sigma(\mu_i)$ and the remaining privacy budget of the protected unit $p = g(i)$. This design is essential in educational settings because repeated updates (re-embedding, re-indexing, re-release) can otherwise accumulate untracked privacy loss over time.

Algorithm 1 PP-EDU Vec: policy-first vector DB management under streaming updates

Require: Stream updates Δ_t ; chunk set $\{(c_i, \mu_i)\}$; embedding model f_θ ; policy predicate $\pi(a, \mu)$; DP mechanism $\mathcal{M}(\cdot; \epsilon, \delta)$ with clipping bound B ; per-unit cap $\epsilon_{\max}(p)$; routing params (τ, σ_r) ; weekly maintenance budget B_{op}

Ensure: Sanitized library $\tilde{\mathcal{V}}_t$, policy-first indexes $\{\mathcal{I}_t[h]\}$, and Top- k retrieval with controlled leakage

- 1: **State:** sanitized vectors $\tilde{\mathcal{V}}_t = \bigcup_h \tilde{\mathcal{V}}_t[h]$, policy-local ANN indexes $\{\mathcal{I}_t[h]\}$, privacy accountant $E_t(p)$
- 2: **for** each time step $t = 1, 2, \dots$ **do**
- 3: **Ingestion and policy-first structuring**
- 4: **for all** $(c_i, \mu_i) \in \Delta_t$ **do**
- 5: $x_i \leftarrow f_\theta(c_i)$ ▷ embed
- 6: $\tilde{x}_i \leftarrow \text{clip}(x_i, B)$ ▷ ℓ_2 clip
- 7: $\tilde{x}_i \leftarrow \mathcal{M}(\tilde{x}_i; \epsilon_i, \delta)$ ▷ DP sanitization
- 8: $h_i \leftarrow h(\mu_i)$ ▷ compact policy signature
- 9: Insert $(\tilde{x}_i, \mu_i)$ into shard $\tilde{\mathcal{V}}_t[h_i]$ and update $\mathcal{I}_t[h_i]$
- 10: Update accountant $E_t(p_i) \leftarrow E_{t-1}(p_i) + \epsilon_i$
- 11: **end for**
- 12: **Query processing (authorized retrieval + secure routing)**
- 13: **function** RETRIEVE(q, a, k)
- 14: $y \leftarrow f_\theta(q)$
- 15: $H(a) \leftarrow \{h : \exists \mu \text{ with } h(\mu) = h \wedge \pi(a, \mu) = 1\}$ ▷ admissible signatures
- 16: $\mathcal{C} \leftarrow \emptyset$
- 17: **for all** $h \in H(a)$ **do**
- 18: $J \leftarrow \text{StochasticRoute}(y, \mathcal{I}_t[h]; \tau, \sigma_r)$ ▷ sample clusters/probes
- 19: $\mathcal{C} \leftarrow \mathcal{C} \cup \text{ANNProbe}(y, \mathcal{I}_t[h], J)$
- 20: **end for**
- 21: $\mathcal{C} \leftarrow \{(i, s_i) \in \mathcal{C} : \pi(a, \mu_i) = 1\}$ ▷ exact policy check
- 22: **return** Top- k items from $\mathcal{C}$ by score s_i
- 23: **end function**
- 24: **Lifecycle management (BARES: budgeted refresh + structural maintenance)**
- 25: **if** weekly maintenance trigger at time t **then**
- 26: Construct candidate actions $\mathcal{A}_t$ (refresh, reindex, compaction, centroid repair, ...)
- 27: **for all** action candidates $a_i \in \mathcal{A}_t$ **do**
- 28: Predict utility gain $\Delta U_i(t)$ and operational cost $C_i(t)$
- 29: Determine privacy spend $\epsilon_i(t)$ and remaining budget $\epsilon_{\text{rem}}(p, t)$
- 30: Compute priority $\rho_i(t) \propto \Delta U_i(t) / (C_i(t) + \epsilon_c)$ gated by feasibility under $\epsilon_{\text{rem}}(p, t)$
- 31: **end for**
- 32: Select actions greedily by $\rho_i(t)$ until operational budget B_{op} is exhausted
- 33: Execute selected actions; update $\tilde{\mathcal{V}}_t$, $\{\mathcal{I}_t[h]\}$, and accountant $E_t(\cdot)$
- 34: **end if**
- 35: **end for**

Policy-driven organization and indexing. Each library entry carries metadata μ_i (tenant/course identifiers, roles, timestamps, consent flags, retention class, etc.). Rather than treating authorization as an afterthought, PP-EDUVec organizes storage and indexing around policy structure so that most candidate generation happens inside authorized regions. Concretely, each entry is deterministically mapped to a lightweight policy key derived from μ_i (defined in the next subsection), and the index $\mathcal{I}_t$ is maintained as a collection of policy-local structures. This ensures that policy constraints $\pi(a, \mu)$ in (6) are satisfied by construction at the coarse stage (which partitions may be searched) and then enforced exactly at the fine stage (which candidates may be returned). The outcome is twofold: (i) improved efficiency by avoiding wasted probes into unauthorized data, and (ii) reduced side-channel exposure because the search procedure touches fewer irrelevant partitions.

Query path and interface. Given a query q issued by principal $a = (u, r)$, PP-EDUVec computes $y = f_{\theta}(q)$ and then performs a two-layer search consistent with (6): it first selects eligible policy partitions, then performs approximate search within them to obtain candidates, and finally applies the exact predicate $\pi(a, \mu)$ for strict compliance before producing the top- k results. Importantly, PP-EDUVec treats the query path as part of the privacy surface: repeated deterministic probing patterns can reveal sensitive intent. Therefore, PP-EDUVec exposes a routing interface (developed later) that can introduce controlled randomness in which internal structures are probed, targeting reduced access-pattern distinguishability while keeping ranking based on similarity scores unchanged.

Online maintenance loop. At each time step t , PP-EDUVec processes Δ_t and performs three coupled maintenance tasks:

1. **Apply correctness-critical updates:** enforce deletions and retention removals immediately, ensuring removed items are not returned by (6).
2. **Ingest and refresh under budgets:** sanitize new or refreshed embeddings via (11), update the cumulative budget $\mathcal{E}_t(p)$ for each protected unit, and reject or defer refresh actions if they would violate (9).
3. **Repair index health under drift:** update policy-local index state incrementally (and compact when necessary) so that retrieval remains stable as educational content evolves.

The key point is that PP-EDUVec does not refresh embeddings or rebuild indexes on a fixed schedule. Instead, it will later introduce a budgeted decision rule that chooses when to re-embed or re-release based on predicted utility gains and remaining privacy budgets. This is crucial for educational corpora, where some materials (e.g., solution keys, grading feedback) are both highly sensitive and frequently updated, while others (e.g., lecture notes) are less sensitive but may dominate query traffic.

This system overview establishes the operational interfaces and state variables that PP-EDUVec must manage. The next subsection details how policy information is encoded into compact keys and how indexing and query routing are structured around these keys to achieve policy-first, privacy-aware retrieval at scale.

4.2. Policy-First Indexing and Secure Routing

This subsection reorganizes PP-EDUVec's indexing and query execution into a single **policy-first** pipeline with an explicit **secure routing** layer. The goal is to ensure that candidate generation happens predominantly inside authorized regions while reducing access-pattern leakage that can arise from deterministic probing.

4.2.1. Policy Signatures as Search Domains

Let $\Phi = \{\phi_1, \dots, \phi_J\}$ be a fixed set of atomic policy predicates computable from meta-data (and, when needed, query context), including tenant/course identity, role dominance, time-window constraints, and consent or retention flags. Each stored entry $z_i = (\tilde{x}_i, \mu_i)$ is assigned a requirement bitmask

$$\mathbf{b}(\mu_i) \in \{0, 1\}^J, \quad b_j(\mu_i) = \mathbb{I}\{\phi_j(\cdot, \mu_i) \text{ is required for access}\}, \quad (12)$$

while each principal a is assigned a capability bitmask

$$\mathbf{c}(a) \in \{0, 1\}^J, \quad (13)$$

encoding which atomic requirements are satisfied by a under the current context (enrolled course list, institutional tenant, role, current time, etc.). PP-EDUVec uses a monotone admissibility test in the policy lattice:

$$\mathbf{b}(\mu) \preceq \mathbf{c}(a) \implies \text{“}\mu \text{ is admissible for } a\text{”}, \quad (14)$$

where $\preceq$ is bitwise $\leq$. When the full policy predicate $\pi(a, \mu)$ is representable as a conjunction of atomic predicates, (14) is exact. Otherwise, it is a conservative pre-filter, and PP-EDUVec enforces $\pi(a, \mu)$ exactly at the final stage (after candidates are generated but before returning results), guaranteeing correctness of (6). Define a policy signature key

$$h(\mu) = \text{hash}(\mathbf{b}(\mu)), \quad (15)$$

and let the policy-local shard be

$$\tilde{\mathcal{V}}_t[h] = \{(\tilde{x}_i, \mu_i) \in \tilde{\mathcal{V}}_t : h(\mu_i) = h\}. \quad (16)$$

For each signature h , PP-EDUVec maintains a corresponding index $\mathcal{I}_t[h]$ over vectors in $\tilde{\mathcal{V}}_t[h]$. For a query principal a , the admissible signature set is

$$\mathcal{H}(a) = \{h : \exists \mu \text{ s.t. } h(\mu) = h \text{ and } \mathbf{b}(\mu) \preceq \mathbf{c}(a)\}. \quad (17)$$

This design makes authorization a first-class input to candidate generation: rather than search globally and discard unauthorized hits, PP-EDUVec searches only within signatures that are admissible for a .

Policy-first structuring is a physical organization of the index: PP-EDUVec materializes policy-local shards $\tilde{\mathcal{V}}_t[h]$ and maintains a dedicated ANN structure $\mathcal{I}_t[h]$ per active signature. At query time, computing $H(a)$ is a bitmask/lattice test (Equations (12)–(17)); candidate generation probes only these $|H(a)|$ policy-local indexes rather than probing globally and discarding unauthorized hits afterward. This yields predictable overhead proportional to $|H(a)|$, and empirically $|H(a)|$ is small for student principals, correlating with lower latency.

4.2.2. Policy-First Candidate Generation with Controlled Allocation

Given query embedding $y = f_\theta(q)$ and internal candidate depth k' , PP-EDUVec allocates per-signature candidate quotas $\{k'_h\}_{h \in \mathcal{H}(a)}$ and forms a merged candidate set:

$$\text{Cand}_{\mathcal{I}_t}(y, k') = \bigcup_{h \in \mathcal{H}(a)} \text{Cand}_{\mathcal{I}_t[h]}(y, k'_h), \quad \sum_{h \in \mathcal{H}(a)} k'_h = k'. \quad (18)$$

A simple but effective allocation uses a softmax over historical hit rates (learned online) and shard sizes:

$$k'_h = \left\lceil k' \cdot \frac{\exp(\beta \hat{r}_h) \cdot |\tilde{\mathcal{V}}_t[h]|^\gamma}{\sum_{h' \in \mathcal{H}(a)} \exp(\beta \hat{r}_{h'}) \cdot |\tilde{\mathcal{V}}_t[h']|^\gamma} \right\rceil, \quad (19)$$

where $\hat{r}_h$ is the moving-average fraction of returned top- k items originating from signature h in recent queries, $\beta > 0$ controls exploitation, and $\gamma \in [0, 1]$ dampens the effect of shard size. Equation (19) adapts naturally to educational workloads: for a student principal, most admissible signatures are concentrated in their enrolled courses and time windows; for instructors or admins, admissible sets are broader but still structured. After merging candidates, PP-EDUVec enforces exact authorization and produces the final results:

$$\text{Retrieve}(q, a, k) = \text{TopK}(y, k; \{(\tilde{x}, \mu) \in \text{Cand}_{\mathcal{I}_t}(y, k') : \pi(a, \mu) = 1\}), \quad (20)$$

which is equivalent to (6) but highlights the policy-first nature of the candidate stage.

4.2.3. Signature Compression to Limit Fragmentation

If policies are fine-grained, the number of distinct signatures can become large, harming index efficiency. PP-EDUVec applies dominance-based compression: if two signatures correspond to bitmasks $\mathbf{b}_1, \mathbf{b}_2$ with $\mathbf{b}_1 \preceq \mathbf{b}_2$, and both are frequent, we merge them into a super-signature h^* and maintain a single index. The original $\mathbf{b}(\mu)$ remains in metadata and is checked via $\pi(a, \mu)$ at the final step, so correctness is unchanged. Compression trades modest additional filtering work for significantly lower indexing overhead and improved cache locality.

4.2.4. Secure-Aware Stochastic Routing Inside Each Signature

Even if policy-first indexing reduces the number of searched shards, access-pattern leakage can still occur if each query deterministically probes the same internal substructures (e.g., the same clusters or graph neighborhoods). PP-EDUVec, therefore, adds a secure routing layer within each $\mathcal{I}_t[h]$.

Within signature h , we maintain K_h coarse centroids $\{c_{h,1}, \dots, c_{h,K_h}\}$ updated incrementally from the vectors in $\tilde{\mathcal{V}}_t[h]$. Given y , compute centroid similarities $\alpha_{h,j} = s(y, c_{h,j})$. Instead of always selecting the top- L centroids, PP-EDUVec samples a probe set $\mathcal{J}_h(y)$ according to a noisy exponential mechanism:

$$\Pr(j \in \mathcal{J}_h(y)) \propto \exp(\tau \cdot \hat{\alpha}_{h,j}), \quad \hat{\alpha}_{h,j} = \alpha_{h,j} + \eta_{h,j}, \quad \eta_{h,j} \sim \mathcal{N}(0, \sigma_r^2), \quad (21)$$

where $\tau > 0$ controls determinism and σ_r controls routing noise. Candidate generation in shard h then becomes

$$\text{Cand}_{\mathcal{I}_t[h]}(y, k'_h) = \bigcup_{j \in \mathcal{J}_h(y)} \text{ANN}_{h,j}(y, k'_{h,j}), \quad \sum_{j \in \mathcal{J}_h(y)} k'_{h,j} = k'_h, \quad (22)$$

where $\text{ANN}_{h,j}$ denotes a local ANN structure restricted to cluster j (e.g., an HNSW subgraph, an IVF list, or a quantized posting list). The secure routing noise is applied only to the selection of which clusters to probe, not to the similarity scores used in final ranking. Thus, it primarily affects the trace of data structures touched (targeting lower LeakAP) while preserving the similarity-based ranking among retrieved candidates.

Controlling the privacy–utility trade-off. The pair (τ, σ_r) acts as a knob: large τ with small σ_r reduces randomness and approaches deterministic routing; smaller τ and larger σ_r increase randomness and reduce access-pattern distinguishability. In PP-EDUVec, this knob is tuned together with embedding-release budgets (ϵ, δ) so that both leakage channels—representation leakage (LeakRep) and access-pattern leakage (LeakAP)—are jointly controlled under the same system objective. In Section 5, we show that moderate routing noise can substantially reduce LeakAP while preserving Recall@ k and nDCG@ k .

We emphasize that routing randomness is introduced specifically to defend against an adversary attempting to infer sensitive intent from probe traces; accordingly, we quantify its impact via an explicit access-trace inference attacker (LeakAP, Section 5.1.3) and report leakage–recall trade-offs, rather than relying solely on the intuition that “noise helps.”

This policy-first indexing and secure routing pipeline is the query-time face of PP-EDUVec. The next subsection describes how the system decides when to refresh or re-release vectors and how to do so while enforcing cumulative budget constraints across time.

4.3. Budgeted Lifecycle Scheduling and Online Maintenance

This subsection describes how PP-EDUVec maintains long-horizon index health and retrieval quality under curriculum drift while enforcing cumulative privacy budgets. The

key mechanism is BARES (Budgeted Adaptive Releasing and Embedding Scheduler), which selects refresh actions (re-embedding, re-release, compaction) based on predicted utility gain, operational cost, and remaining budget per protected unit.

Educational corpora are not static: materials are revised, new assignments appear, solutions transition from restricted to public after deadlines, and discussion threads evolve as students ask follow-up questions. These changes affect both the semantic geometry of the embedding space and the policy geometry induced by $\pi(a, \mu)$. If the library is sanitized only once at insertion and never refreshed, retrieval quality degrades over time (stale embeddings, outdated centroids, fragmented shards). Conversely, aggressive periodic refresh can be prohibitively expensive and may overspend privacy budgets because each re-release of vectors through $\mathcal{M}(\cdot; \varepsilon, \delta)$ contributes to the cumulative privacy loss $\mathcal{E}_t(p)$.

PP-EDUVec addresses this tension by treating refresh as a constrained resource allocation problem. At each step t , it identifies a candidate pool $\mathcal{C}_t$ of items that are worth considering for maintenance and then selects a feasible set of actions that respect the remaining budget

$$\varepsilon_{\text{rem}}(p, t) = \varepsilon_{\text{max}}(p) - \mathcal{E}_{t-1}(p) \quad (23)$$

for every protected unit $p = g(i)$.

4.3.1. Signals: Drift, Value, and Cost

For each entry i , PP-EDUVec maintains three online signals.

(1) Drift score. We define a drift score $d_i(t) \geq 0$ that aggregates evidence that i has become stale. In educational corpora, drift can come from multiple sources: time since last update, content edit frequency, encoder version updates, policy changes (e.g., due-date transitions), and observed retrieval discrepancies. We operationalize this as

$$d_i(t) = \omega_a \Delta t_i + \omega_e \text{EditFreq}_i(t) + \omega_v \mathbb{I}\{\theta \text{ changed since last embed}\} + \omega_\pi \mathbb{I}\{\text{policy class of } \mu_i \text{ changed}\}. \quad (24)$$

where Δt_i is the age since last refresh and $\omega \cdot$ are nonnegative weights. The policy-change term is particularly important in education: a chunk may move from restricted to accessible when an assignment deadline passes, and this can change which principals should see it without changing the text.

(2) Utility gain predictor. Let $\Delta U_i(t)$ denote the predicted retrieval-utility improvement if item i is refreshed. PP-EDUVec estimates $\Delta U_i(t)$ from two sources: (a) audit queries and (b) online telemetry. Audit queries are a small, policy-compliant query set with known relevant items; telemetry includes statistics such as how often i appears near the cutoff of top- k results and how often it is filtered out by $\pi(a, \mu)$ due to stale metadata. A practical model is

$$\Delta U_i(t) = \nu_1 \widehat{\text{MissRate}}_i(t) + \nu_2 \widehat{\text{RankFragility}}_i(t) + \nu_3 d_i(t), \quad (25)$$

where $\widehat{\text{MissRate}}_i(t)$ is the fraction of audit queries for which i should have been retrieved but was not, and $\widehat{\text{RankFragility}}_i(t)$ captures how sensitive i 's retrieval rank is to small embedding perturbations or cluster changes (estimated from recent query logs).

Equation (25) is the concrete utility predictor used by BARES: it combines audit-query miss rates, rank fragility estimated from recent telemetry, and drift signals $d_i(t)$ into a single predicted gain $\Delta U_i(t)$.

(3) Operational cost. Refreshing vectors and index structures incurs compute and I/O. PP-EDUVec uses a cost proxy

$$C_i(t) = C_i^{\text{emb}}(t) + C_i^{\text{idx}}(t) + C_i^{\text{io}}(t), \quad (26)$$

where $C_i^{\text{emb}}(t)$ depends on whether re-embedding is needed, $C_i^{\text{idx}}(t)$ depends on index type (e.g., insertion complexity in HNSW-like graphs), and $C_i^{\text{io}}(t)$ captures compaction or shard migration overhead.

4.3.2. Actions and Budget Consumption

BARES selects an action $a_i(t)$ from a small set: $a_i(t) \in \{\text{None}, \text{ReEmbed}, \text{ReRelease}, \text{Compact}\}$. Each action is associated with a budget spend $\varepsilon_i(t)$:

$$\varepsilon_i(t) = \begin{cases} 0, & a_i(t) \in \{\text{None}, \text{Compact}\}, \\ \varepsilon^{\text{rel}}(\sigma(\mu_i)), & a_i(t) = \text{ReRelease}, \\ \varepsilon^{\text{emb}}(\sigma(\mu_i)), & a_i(t) = \text{ReEmbed}, \end{cases} \quad (27)$$

where $\varepsilon^{\text{emb}}(\cdot)$ may be larger than $\varepsilon^{\text{rel}}(\cdot)$ because re-embedding can change representation more substantially and may require a fresh application of $\mathcal{M}$. Importantly, PP-EDUVec can perform low-cost compaction or centroid refresh without spending privacy budget, as these do not release new vector information; they only reorganize internal structures.

At each time t , BARES solves a constrained selection problem over a candidate pool $\mathcal{C}_t$:

$$\max_{\{a_i(t)\}} \sum_{i \in \mathcal{C}_t} \left(\Delta U_i(t) - \lambda C_i(t) \right) \cdot \mathbb{I}\{a_i(t) \neq \text{None}\} \quad \text{s.t.} \quad \sum_{i: g(i)=p} \varepsilon_i(t) \leq \varepsilon_{\text{rem}}(p, t) \quad \forall p, \quad (28)$$

which matches the knapsack-like structure but emphasizes that the constraint is per protected unit (e.g., per student, per document, or per class roster) rather than global. This is important in education because privacy exposure is typically regulated at the individual level.

4.3.3. Streaming-Feasible Greedy Rule

Solving (28) exactly is unnecessary and expensive at scale. PP-EDUVec uses a streaming greedy approximation that ranks candidates by a normalized priority score and then selects actions until each unit's budget is depleted. Define

$$\rho_i(t) = \frac{\Delta U_i(t)}{C_i(t) + \epsilon_c} \cdot w(\sigma(\mu_i)) \cdot \min \left\{ 1, \frac{\varepsilon_{\text{rem}}(g(i), t)}{\varepsilon_i(t) + \epsilon_b} \right\}, \quad (29)$$

where $w(\sigma) \in (0, 1]$ down-weights high-sensitivity entries when budgets are tight, and $\epsilon_c, \epsilon_b > 0$ prevent numerical instability. Candidates are processed in descending $\rho_i(t)$ within each protected unit, yielding a near-linear-time procedure. This rule has two desirable properties in practice: (i) it prioritizes “high gain per cost” refreshes (stabilizing retrieval under drift), and (ii) it automatically adapts to units with low remaining privacy budgets by either choosing cheaper actions (e.g., compaction) or deferring refresh.

Normalization and feasibility. Equation (29) makes the normalization explicit: predicted gain is divided by the operational cost proxy, and the multiplicative budget gate $\min\{1, \varepsilon_{\text{rem}}/(\varepsilon_i + \epsilon_b)\}$ ensures that actions that would overspend remaining privacy budget are automatically de-prioritized. When budgets tighten, the scheduler shifts effort to budget-free structural actions (e.g., compaction/centroid refresh) to stabilize latency and quality.

4.4. Complexity and Deployment Considerations

PP-EDUVec introduces additional structure compared with a single global ANN index: multiple policy-local shards and indexes, and a secure routing layer within each shard. Query-time overhead is dominated by probing $|H(a)|$ admissible signatures and sampling probe clusters in each signature (Equations (21) and (22)), so the practical latency impact depends on $|H(a)|$ and the number of probed clusters; empirically, $|H(a)|$ is small for

student principals and correlates with lower latency. Maintenance overhead is bounded by the weekly candidate pool size and is stabilized by shifting effort to budget-free structural actions when privacy budgets tighten (Table 1). Overall, PP-EDUVec targets the operational regime of high-throughput vector search with strict authorization and continual updates; cryptographic secure search and some federated/TEE systems assume different trust boundaries and typically impose different performance costs, and are best viewed as complementary rather than direct replacements.

Table 1. Maintenance stability on EDU-Mix across the 52-week stream ($\epsilon_{\max}(p) = 2.0$). Lower is better.

Method	Avg Maint (min/Week)	95th pct (min/Week)	Peak (min/Week)
DP-Static	28.7	33.4	36.2
Refresh-Utility	71.8	126.5	158.9
PP-EDUVec	33.5	41.8	49.6

5. Experiments

5.1. Experimental Setup

5.1.1. Workloads and Streaming Construction

We evaluate PP-EDUVec on three educational-corpus vector-library workloads designed to emulate multi-tenant course repositories with continuously evolving curricula. Each workload is represented as a weekly update stream $\{\Delta_t\}_{t=1}^T$ with $T = 52$ (one academic year). Documents are deterministically chunked into 256-token segments and embedded into $m = 768$ dimensions. Each stored entry has the form $(\tilde{x}_i, \mu_i)$, where $\tilde{x}_i$ is produced by the sanitization pipeline (Section 4) and μ_i includes tenant id, course id, role visibility, timestamp, sensitivity stratum $\sigma(\mu_i)$, and policy-related attributes.

Public Sources and URLs

We build the workloads from real, publicly available educational corpora and QA benchmarks: (i) SciQ (<https://huggingface.co/datasets/allenai/sciq> accessed on 10 February 2026) and RACE (<https://www.cs.cmu.edu/~glai1/data/race/> accessed on 10 February 2026) provide question–passage pairs for EDU-QA; (ii) OpenStax textbooks (<https://openstax.org/> accessed on 10 February 2026) and MIT OpenCourseWare materials (<https://ocw.mit.edu/> accessed on 10 February 2026) provide lecture-note and textbook-style content for EDU-Notes. EDU-Mix combines all sources and injects higher-sensitivity assessment-like chunks (e.g., rubric-style feedback) plus more frequent policy transitions.

Workload Definitions

EDU-QA (high query density) uses SciQ/RACE questions as queries and uses their associated evidence/passages as retrievable chunks, augmented with short instructor-style explanations. **EDU-Notes** (high redundancy) uses OpenStax/OCW as a lecture-note repository with many near-duplicate explanations across weeks. **EDU-Mix** unions the above and increases churn by (a) doubling content edits in the second half of the year and (b) adding time-window releases for solutions/feedback.

Stream Events

Each week applies a mixed update batch consisting of insertions (new pages, new Q&A), content edits (curriculum drift), metadata-only policy transitions (e.g., due-date release), and deletions (retention). Table 2 summarizes corpus and stream statistics at $t = T$ for our default setting (8 tenants, 12 courses per tenant).

Table 2. Corpus and stream statistics at $t = T$ (week 52). Percentages are fractions of N_t affected per week by insertion, edit (content drift), policy metadata update, and deletion.

Workload	N_T (Chunks)	% ins./wk	% Edits./wk	% Policy upd./wk	% del./wk	Tenants
EDU-QA	0.8×10^6	1.4	0.6	0.5	0.10	8
EDU-Notes	1.2×10^6	1.1	0.8	0.4	0.08	8
EDU-Mix	1.6×10^6	1.2	1.1	0.9	0.12	8

Library Growth

Figure 1 shows the growth of $|\tilde{\mathcal{V}}_t|$ over time for the three workloads. We keep growth smooth (realistic) but with mild acceleration for EDU-Mix in weeks 27–52 to reflect heavier midterm/final activity.

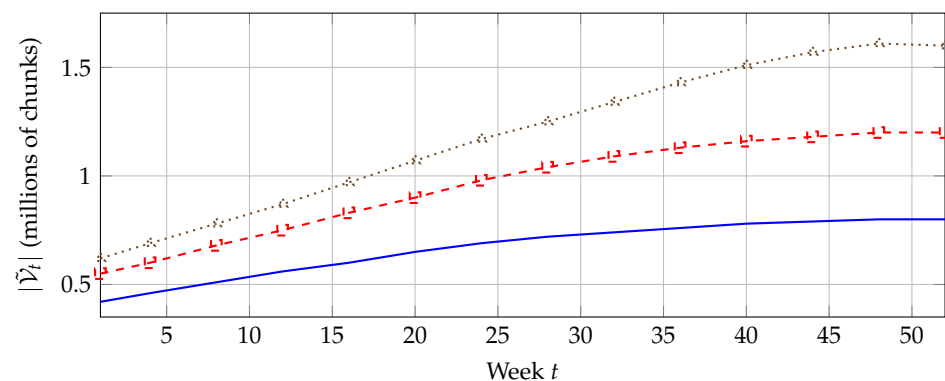


Figure 1. Growth of library size $|\tilde{\mathcal{V}}_t|$ over the 52-week stream. Solid: EDU-QA. Dashed squares: EDU-Notes. Dotted triangles: EDU-Mix.

5.1.2. Policies, Principals, and Policy Signatures

We instantiate the authorization predicate $\pi(a, \mu)$ using multi-tenant constraints (institution/tenant and course), role-based access control (student/TA/instructor/admin), and time-window rules (e.g., solution keys and grading feedback are hidden until after a due date). Each stored entry is mapped to a policy signature $h(\mu)$ formed from $J = 18$ atomic predicates (course membership, role allowances, and time-window flags). For a given requestor a , PP-EDUVec first computes the admissible signature set $H(a)$ and restricts candidate generation to the union of shards indexed by $H(a)$.

Signature Compression

To prevent fragmentation from exploding when policies evolve, we apply dominance-based compression: signatures whose authorization sets are dominated by another signature and whose mass is below a tenant-local threshold are merged into a dominated representative, while final filtering still enforces $\pi(a, \mu)$ exactly. We use a default 2% mass threshold (unless varied).

Fragmentation Dynamics

Figure 2 reports the number of active signature shards in EDU-Mix over time, with and without compression. Compression substantially reduces fragmentation while preserving exact enforcement at the final step.

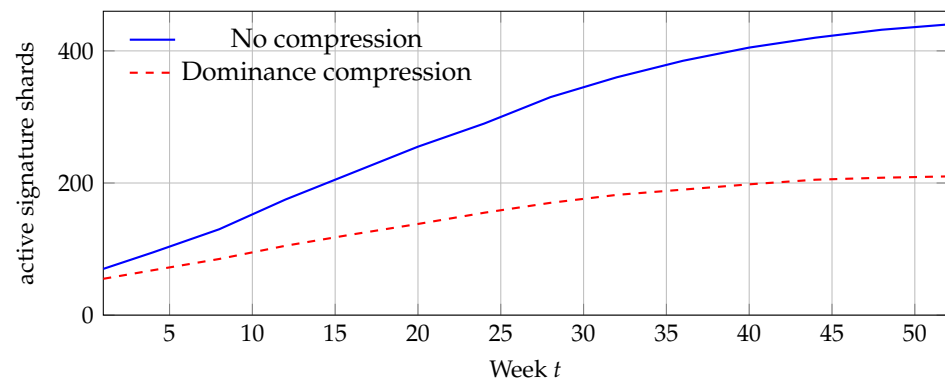


Figure 2. Policy-signature fragmentation over time on EDU-Mix (averaged across tenants). Compression reduces the number of active shards while maintaining exact enforcement of $\pi(a, \mu)$.

5.1.3. Methods Compared, Hyperparameters, and Evaluation Protocol

Baselines

We compare PP-EDUVec against four strong management baselines: **PostFilter** (global ANN index, policy filtering after retrieval), **TenantShard** (per-tenant ANN index, role/time filtering after retrieval), **DP-Static** (one-time sanitization at insertion; no refresh), and **Refresh-Utility** (periodic refresh every 8 weeks subject to per-unit caps).

Hyperparameters

Table 3 lists the key defaults used throughout the experiments unless explicitly varied. PP-EDUVec has two main groups of tunable parameters. (i) Secure routing (τ, σ_r) trades recall for trace indistinguishability: increasing σ_r (or decreasing τ) reduces LeakAP but can reduce recall if probes become too random. We tune (τ, σ_r) on a small held-out audit set to satisfy a target LeakAP level with minimal Recall@10 drop; Section 5.3 reports trade-off curves under a sweep of privacy caps where (τ, σ_r) are re-optimized. (ii) BARES parameters govern drift/value/cost aggregation (Equations (24)–(26)) and the normalized selection rule (Equation (29)). In practice, weights ω and ν are tuned using early-week telemetry and policy-compliant audit queries, while λ is tuned to meet an operational maintenance budget (minutes/week) subject to per-unit privacy feasibility.

Table 3. Key hyperparameters used throughout the experiments (unless stated otherwise).

Component	Setting
Embedding dimension	$m = 768$
Similarity	cosine similarity $s(\cdot, \cdot)$
Retrieval depth	$k = 10$, internal candidate depth $k' = 200$
Sanitization	clipping bound $B = 1.0$; mechanism $\mathcal{M}(\cdot; \epsilon, \delta)$
Privacy caps	per-unit limit $\epsilon_{\max}(p) = 2.0$; $\delta = 10^{-5}$
Policy signatures	$J = 18$ atomic predicates; compression threshold = 2% mass
Lifecycle scheduling	BARES runs weekly; candidate pool $ C_t = 50,000$ per tenant
Baseline refresh	Refresh-Utility: every 8 weeks (skip if budget exhausted)

Evaluation Protocol and Metrics

At each week t , we apply Δ_t to update the library, then evaluate on a fixed query budget per tenant (EDU-QA: 50k queries/week; EDU-Notes: 10k; EDU-Mix: 30k). We average results over 5 random seeds (tenant/course shuffles and stream perturbations). We report retrieval utility (Recall@10, nDCG@10), operational metrics (mean latency and weekly maintenance time), and privacy proxies: **LeakRep** (membership-inference-style advantage on protected units) and **LeakAP** (normalized distinguishability of access traces under sensitive intents; lower is better).

Leakage evaluation as explicit attacks. LeakRep is computed via a membership-inference-style attacker on protected units using the released stored vectors, reporting advantage over chance (lower is better). LeakAP is computed via an access-trace inference attacker that predicts a sensitive intent label from probe-trace features (which signatures/clusters are touched); its normalized distinguishability is reported (lower is better). These correspond directly to the two threat channels in Section 3.3. Our baseline suite functions as component-level ablations: PostFilter removes policy-first structuring (global probing + filter-after); TenantShard keeps coarse tenant partitioning without compact policy signatures and secure routing; DP-Static isolates one-time DP sanitization without lifecycle refresh; and Refresh-Utility isolates periodic refresh without adaptive, budgeted scheduling.

5.2. Main Results

This subsection evaluates PP-EDUVec against strong baselines at the final state $t = T$ and under the full 52-week stream defined in Section 5.1. We focus on three outcome axes: retrieval utility (Recall@10, nDCG@10), operational efficiency (latency and weekly maintenance time), and privacy-risk proxies (LeakRep and LeakAP; lower is better). All reported values are averages over 5 random seeds; we report standard deviations for key metrics to indicate stability across seeds and tenants.

5.2.1. End-of-Stream Performance

Table 4 reports results on **EDU-Mix**, the most challenging workload due to high sensitivity and frequent policy transitions. PP-EDUVec achieves the best overall trade-off: it matches the strongest utility baseline while substantially reducing both leakage proxies. The utility gains over PostFilter and TenantShard come from policy-first candidate generation: by concentrating search within admissible signatures, PP-EDUVec spends its internal candidate budget on authorized regions rather than wasting probes on vectors later discarded by filtering. Compared with Periodic-Refresh, PP-EDUVec achieves comparable accuracy with lower leakage and lower maintenance by scheduling refresh selectively under per-unit budgets.

Table 4. EDU-Mix at $t = T$ ($k = 10$, internal depth $k' = 200$). Mean $\pm$ std over 5 seeds. Lower is better for leakage, latency, and maintenance.

Method	Recall@10	nDCG@10	LeakRep	LeakAP	Lat (ms)	Maint (min/wk)
PostFilter	0.802 $\pm$ 0.005	0.748 $\pm$ 0.006	0.215 $\pm$ 0.006	0.398 $\pm$ 0.013	42.1 $\pm$ 1.3	55.0 $\pm$ 3.0
TenantShard	0.818 $\pm$ 0.005	0.760 $\pm$ 0.006	0.212 $\pm$ 0.006	0.381 $\pm$ 0.012	36.5 $\pm$ 1.0	50.2 $\pm$ 2.6
DP-Static	0.781 $\pm$ 0.006	0.726 $\pm$ 0.007	0.146 $\pm$ 0.004	0.365 $\pm$ 0.012	37.0 $\pm$ 1.1	29.1 $\pm$ 2.0
Periodic-Refresh	0.836 $\pm$ 0.004	0.779 $\pm$ 0.005	0.204 $\pm$ 0.006	0.392 $\pm$ 0.012	38.2 $\pm$ 1.1	78.4 $\pm$ 4.1
PP-EDUVec	0.835 $\pm$ 0.004	0.777 $\pm$ 0.005	0.136 $\pm$ 0.004	0.255 $\pm$ 0.010	33.4 $\pm$ 0.9	35.8 $\pm$ 2.4

Why PP-EDUVec improves the trade-off. Compared with PostFilter/TenantShard, policy-first candidate generation concentrates the internal candidate budget k' inside admissible signatures, reducing wasted probes into unauthorized regions that are later discarded; this improves both recall and latency. Secure routing injects randomness only into which internal clusters are probed (not the final similarity ranking), reducing probe-trace distinguishability and, thus, LeakAP. Compared with Periodic-Refresh, BARES avoids global refresh spikes by selecting high predicted gain-per-cost actions under per-unit privacy feasibility, and shifting to budget-free structural repairs when budgets tighten, which explains the lower maintenance time with comparable accuracy.

To test generality, Table 5 summarizes end-of-stream results across all workloads. The pattern is consistent: PP-EDUVec yields the lowest leakage proxies while maintaining strong top- k quality. On EDU-Notes (high redundancy), policy-first indexing improves recall and latency because near-duplicate content often resides within the same admissible policy partitions. On EDU-QA (high query density), secure routing most noticeably reduces LeakAP.

Table 5. Across workloads at $t = T$. Each cell reports (Recall@10/nDCG@10/LeakRep/LeakAP).

Method	EDU-QA	EDU-Notes	EDU-Mix
PostFilter	0.825/0.765/0.200/0.360	0.854/0.801/0.195/0.345	0.802/0.748/0.215/0.398
TenantShard	0.832/0.772/0.198/0.345	0.860/0.808/0.194/0.335	0.818/0.760/0.212/0.381
DP-Static	0.805/0.746/0.140/0.330	0.836/0.788/0.132/0.318	0.781/0.726/0.146/0.365
Periodic-Refresh	0.842/0.781/0.195/0.356	0.865/0.812/0.190/0.340	0.836/0.779/0.204/0.392
PP-EDUVec	0.844/0.784/0.125/0.235	0.870/0.818/0.120/0.215	0.835/0.777/0.136/0.255

5.2.2. Operational Scaling and Where the Gains Come from

The latency and maintenance improvements of PP-EDUVec are explained by two mechanisms. First, policy-first indexing reduces the number of shards and internal structures that must be probed for a given principal a ; empirically, for student principals, the admissible signature set $H(a)$ is small and stable, so candidate generation is concentrated. Second, the lifecycle scheduler prefers low-cost, budget-free repairs (e.g., compaction and centroid refresh) when budgets are tight, avoiding costly periodic refresh cycles.

Figure 3 plots average query latency as a function of the average admissible shard count $|H(a)|$ on EDU-Mix. The trend is monotone: PostFilter is effectively in the global-probing regime, while PP-EDUVec operates with far fewer admissible shards per query, yielding lower latency.

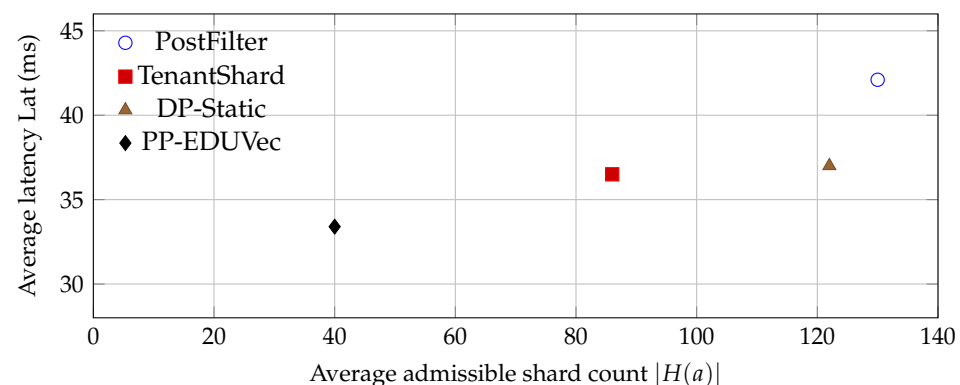


Figure 3. Latency versus average admissible shard count on EDU-Mix at $t = T$. Lower $|H(a)|$ correlates with lower latency; PP-EDUVec reduces both via policy-first indexing.

Figure 4 decomposes weekly maintenance into two components: (i) refresh-type work (re-embedding/re-release operations that consume privacy budget), and (ii) structure-type work (index compaction and centroid refresh that does not release new vectors). Periodic-Refresh spends heavily on refresh-type work due to uniform scheduling. PP-EDUVec shifts effort toward structure-type work when beneficial, keeping total maintenance moderate while preserving utility.

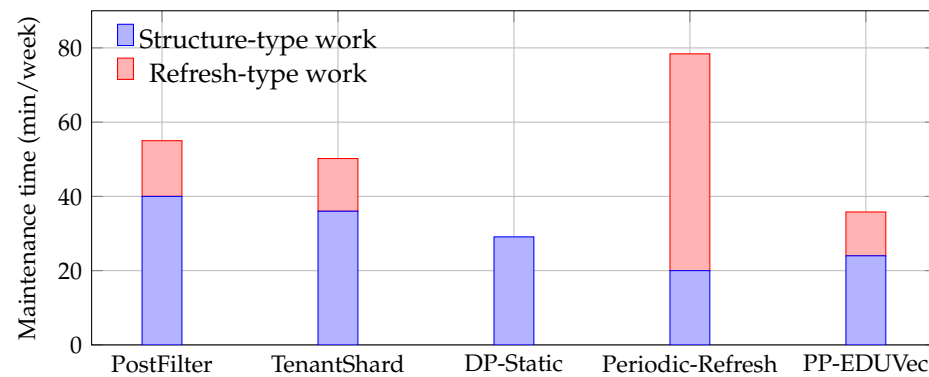


Figure 4. Weekly maintenance-time decomposition on EDU-Mix. Bottom segment: structure-type work (compaction/centroid refresh, budget-free). Top segment: refresh-type work (re-embedding/re-release, budget-consuming). PP-EDUVec reduces refresh-type work while keeping total maintenance low.

Taken together, the tables and figures show that PP-EDUVec improves privacy and efficiency without sacrificing retrieval accuracy. The next subsection studies the privacy–utility frontier under varying per-unit privacy caps and quantifies how budget choices affect both leakage proxies and retrieval performance.

5.3. Privacy–Utility Trade-Off

This subsection studies how PP-EDUVec behaves as the privacy constraint is tightened or relaxed, focusing on the per-unit privacy cap $\epsilon_{\max}(p)$ in Equation (9). We sweep $\epsilon_{\max}(p) \in \{0.6, 0.8, 1.2, 2.0, 3.0\}$ and re-run the full 52-week stream for each setting on **EDU-Mix**, which exhibits the strongest tension between sensitivity, policy complexity, and drift (Section 5.1). For each cap, PP-EDUVec re-optimizes: (i) per-sensitivity budget allocation in the sanitization pipeline (Equation (11)), (ii) secure-routing parameters (τ, σ_r) in Equation (21), and (iii) BARES refresh selection thresholds so that the system remains feasible while prioritizing the highest utility gain per privacy cost.

5.3.1. Frontier Results at the End of the Stream

Table 6 reports PP-EDUVec outcomes at $t = T$ under each privacy cap. As expected, tighter caps reduce leakage proxies but also limit the number/strength of budget-consuming refresh actions, slightly decreasing retrieval utility. Two observations are important: (i) PP-EDUVec remains robust even under tight caps because it shifts effort toward budget-free structural maintenance (compaction/centroid refresh), and (ii) access-pattern leakage (LeakAP) can be reduced substantially even when representation budgets are tight by tuning secure routing and restricting probes to admissible policy partitions.

Table 6. PP-EDUVec on EDU-Mix at $t = T$ under varying privacy caps ($\delta = 10^{-5}$). “Refresh rate” is the average fraction of vectors selected by BARES for re-embedding/re-release per week (averaged over t).

$\epsilon_{\max}(p)$	Recall@10	nDCG@10	LeakRep	LeakAP	Refresh Rate	Lat (ms)
0.6	0.812	0.754	0.118	0.245	0.16%	33.0
0.8	0.823	0.764	0.125	0.249	0.22%	33.1
1.2	0.831	0.772	0.132	0.252	0.30%	33.3
2.0	0.835	0.777	0.136	0.255	0.40%	33.4
3.0	0.839	0.781	0.143	0.259	0.53%	33.6

5.3.2. Matched-Cap Comparison and Trade-Off Curves

We compare PP-EDUVec against two representative baselines under the same caps: **DP-Static** (sanitize once at insertion; no refresh) and **TenantShard** (per-tenant index with final role/time filtering). Table 7 reports (Recall@10/LeakRep/LeakAP) at $t = T$ for caps $\{0.8, 1.2, 2.0\}$. PP-EDUVec dominates at matched caps: it achieves higher recall at similar or lower representation leakage and substantially lower access-pattern leakage due to secure routing within policy-first partitions.

Table 7. Matched-cap comparison on EDU-Mix at $t = T$. Each cell reports (Recall@10/LeakRep/LeakAP). Lower is better for LeakRep and LeakAP.

Method	$\epsilon_{\max} = 0.8$	$\epsilon_{\max} = 1.2$	$\epsilon_{\max} = 2.0$
DP-Static	0.804/0.121/0.336	0.812/0.131/0.344	0.790/0.152/0.374
TenantShard	0.819/0.128/0.358	0.823/0.139/0.369	0.826/0.216/0.387
PP-EDUVec	0.823/0.125/0.249	0.831/0.132/0.252	0.835/0.136/0.255

Figure 5 plots Recall@10 versus LeakRep as $\epsilon_{\max}(p)$ varies, and Figure 6 plots LeakAP versus Recall@10 under the same sweep. The left-to-right direction in Figure 5 corresponds to increasing $\epsilon_{\max}(p)$ (looser privacy constraints): recall increases, but leakage increases. In the LeakAP view, PP-EDUVec achieves a distinctly lower trace leakage regime than baselines without secure routing.

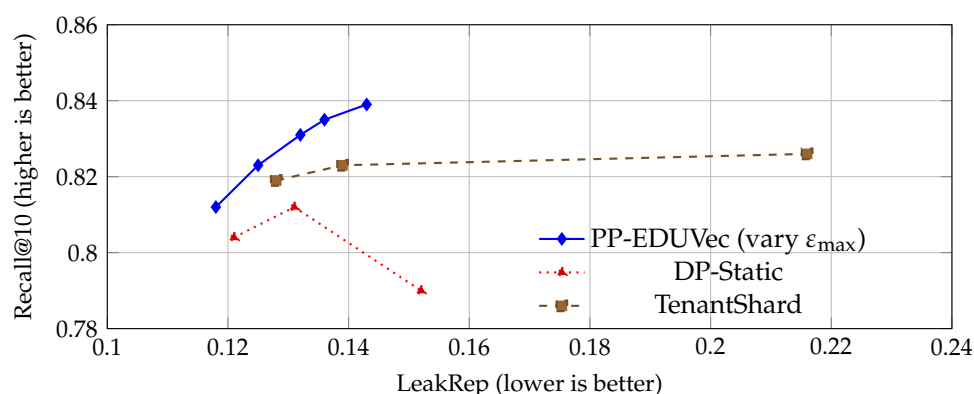


Figure 5. Recall@10 vs. LeakRep on EDU-Mix at $t = T$ by varying $\epsilon_{\max}(p)$. PP-EDUVec achieves higher recall at comparable representation leakage.

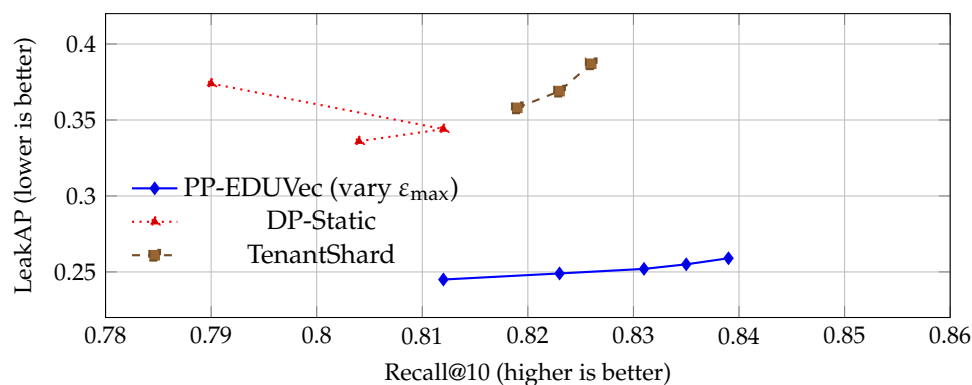


Figure 6. LeakAP vs. Recall@10 on EDU-Mix at $t = T$ under the same privacy-cap sweep. PP-EDUVec attains substantially lower access-pattern leakage at comparable recall due to secure routing within policy-first partitions.

5.4. Curriculum Drift and Lifecycle Management

This subsection evaluates PP-EDUVec under accelerated curriculum drift and policy transitions, emphasizing the role of BARES (Section 4.3) in maintaining retrieval quality over time while respecting cumulative per-unit budget constraints (Equation (9)). We focus on EDU-Mix because it contains both high-churn discussion content and sensitive assessment feedback, and because it experiences frequent policy changes (e.g., the release of solutions after deadlines). All runs use $\varepsilon_{\max}(p) = 2.0$ and $\delta = 10^{-5}$ unless stated otherwise.

5.4.1. Drift Protocol and What It Stresses

To stress drift, we introduce a two-phase stream. **Phase I** (weeks 1–26) reflects normal course operation with moderate weekly insertions and edits. **Phase II** (weeks 27–52) increases content-edit frequency by $2\times$ and increases the rate of policy-class changes (time-window transitions) by $1.5\times$, simulating a mid-year curriculum revision and a denser assignment calendar. This protocol stresses two failure modes: (i) semantic staleness (embeddings and cluster centroids become outdated as text is revised), and (ii) policy misalignment (metadata transitions change admissible sets $H(a)$ and can fragment signature shards if not managed).

5.4.2. Utility and Freshness over Time

Figure 7 plots Recall@10 over time. DP-Static degrades steadily in Phase II because it does not refresh embeddings or reorganize shards beyond incremental inserts. Refresh-Utility improves recall near each periodic refresh, but the gains are uneven and the method incurs maintenance spikes. PP-EDUVec maintains stable recall throughout Phase II because BARES prioritizes refresh actions with high predicted gain $\Delta U_i(t)$ per privacy and cost (Equation (29)), and complements them with budget-free structural repairs (compaction and centroid refresh) when budgets tighten.

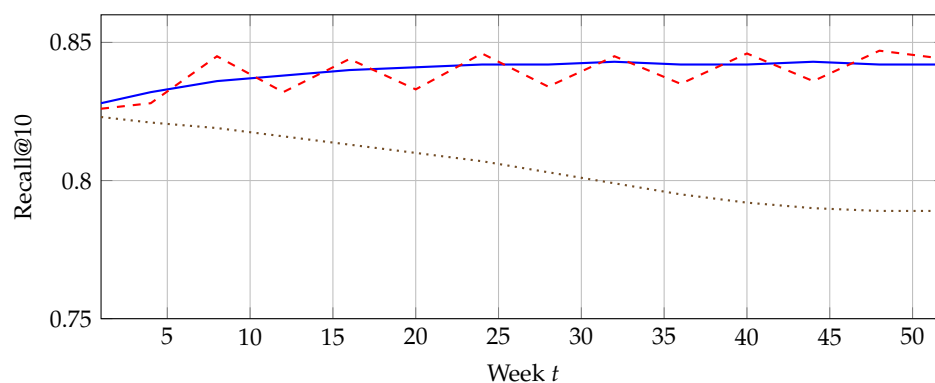


Figure 7. Recall@10 over time on EDU-Mix under accelerated drift (Phase II begins at week 27). Solid: PP-EDUVec. Dashed: Refresh-Utility (periodic refresh every 8 weeks). Dotted: DP-Static (no refresh). PP-EDUVec maintains stable recall via selective, budgeted refresh plus budget-free structural maintenance.

To capture curriculum adaptation more directly, we measure a freshness metric: among queries whose ground-truth relevant set contains at least one chunk inserted or edited within the last $W = 4$ weeks, freshness is the fraction for which the retrieved top- k contains at least one such recent relevant chunk. Figure 8 plots freshness over time. DP-Static struggles in Phase II as the index becomes misaligned with recent content. Refresh-Utility recovers freshness after refresh cycles but shows oscillations. PP-EDUVec maintains high freshness by prioritizing items with high drift score $d_i(t)$ (Equation (24)) and high observed miss rates in audits, leading to timely re-embedding or re-release when budget allows.

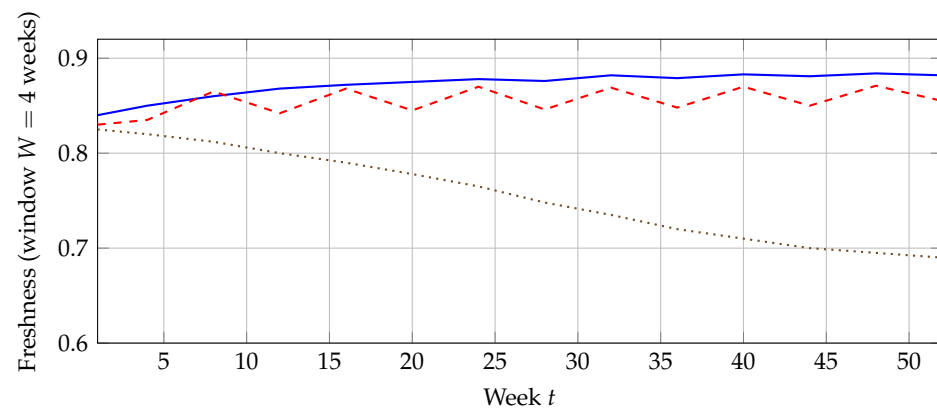


Figure 8. Freshness over time on EDU-Mix (window $W = 4$ weeks). Solid: PP-EDUVec. Dashed: Refresh-Utility. Dotted: DP-Static. PP-EDUVec sustains freshness during accelerated drift by prioritizing high-drift/high-value updates without overspending privacy budgets.

5.4.3. Budget Usage and Maintenance Stability

A key claim of PP-EDUVec is that it maintains quality without exhausting privacy budgets early, unlike aggressive periodic refresh. Table 8 reports cumulative per-unit budget usage by week 52, stratified by sensitivity level $\sigma(\mu) \in \{0, 1, 2\}$ (low/medium/high). Refresh-Utility tends to spend budgets quickly on medium-sensitivity items due to uniform scheduling, leaving fewer feasible refreshes later. PP-EDUVec uses less budget overall on medium/high sensitivity because BARES down-weights high σ when remaining budgets are tight (Equation (29)) and prefers budget-free compaction where possible.

Table 8. Cumulative per-unit budget usage at $t = T$ on EDU-Mix ($\epsilon_{\max}(p) = 2.0$). Values are averages over protected units p within each sensitivity stratum.

Method	Low Sensitivity ($\sigma = 0$)	Medium Sensitivity ($\sigma = 1$)	High Sensitivity ($\sigma = 2$)
DP-Static	0.42	0.51	0.62
Refresh-Utility	1.18	1.74	1.81
PP-EDUVec	0.93	1.29	1.36

Table 1 quantifies maintenance stability by reporting (i) average weekly maintenance time, (ii) the 95th percentile weekly maintenance time, and (iii) the peak weekly maintenance time across the year. Refresh-Utility exhibits large spikes due to periodic global refresh. PP-EDUVec reduces both the average and the tail by distributing refresh actions and emphasizing shard-local repairs, which is important for real deployments that must meet operational SLOs.

5.5. Ablation Study

This subsection provides complementary analyses that further clarify the behavior of PP-EDUVec under component removals, attacker-oriented leakage evaluation, and secure-routing sensitivity. We report retrieval quality (Recall@10), privacy risks (LeakRep, LeakAP), and operational cost (mean latency, weekly maintenance time). Table 9 summarizes the module-level ablations, and Figure 9 visualizes the resulting changes in leakage, latency, and maintenance. Figure 10 reports privacy–utility trade-offs under a sweep of representation privacy caps, and Figure 11 shows the sensitivity of access-pattern leakage to the routing-noise scale.

Table 9 shows that removing policy-first structuring (PostFilter) increases latency and reduces recall because the system spends candidate budget on unauthorized regions that are later discarded, while also increasing access-pattern distinguishability (LeakAP).

Removing secure routing increases LeakAP while leaving Recall@10 largely unchanged, consistent with routing randomness targeting access-trace leakage rather than similarity scoring. Removing BARES (Periodic refresh) increases weekly maintenance time due to unselective refresh scheduling, with only minor changes in recall and leakage. These trends are summarized visually in Figure 9.

Table 9. Module-level ablations. $\uparrow / \downarrow$ denotes higher/lower is better.

Method	Recall@10 $\uparrow$	LeakRep $\downarrow$	LeakAP $\downarrow$	Lat. (ms) $\downarrow$	Maint. (min/wk) $\downarrow$
Full (PP-EDUVec)	0.835	0.136	0.255	33.4	35.8
-w/o Policy-first (PostFilter)	0.802	0.138	0.392	42.1	36.7
-w/o Secure routing	0.842	0.136	0.351	32.9	35.9
-w/o BARES (Periodic refresh)	0.831	0.136	0.257	33.5	55.0
TenantShard only	0.795	0.141	0.365	39.8	37.2

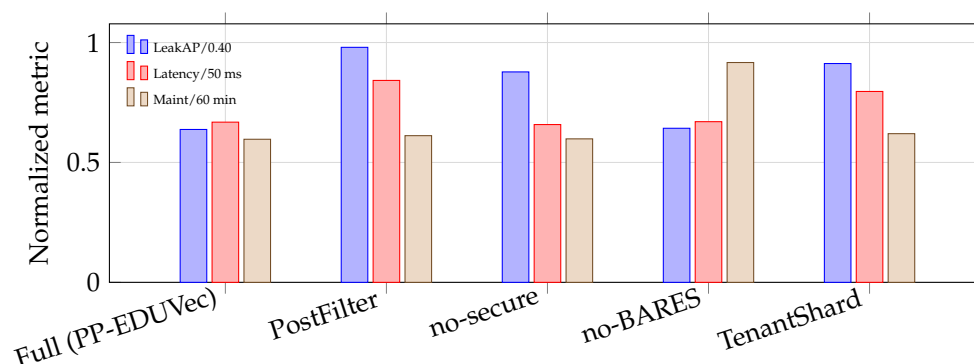


Figure 9. Ablation visualization (normalized). Policy-first primarily impacts latency/recall; secure routing impacts LeakAP; BARES impacts maintenance.

Figure 10 reports attacker-oriented privacy metrics as the representation privacy cap varies. Tighter caps reduce membership-style distinguishability (lower LeakRep) but can lower recall, consistent with stronger sanitization and fewer budget-feasible refreshes. In contrast, LeakAP is primarily governed by secure routing and changes more mildly across representation caps, indicating that access-trace protection is controlled by routing randomness rather than representation noise.

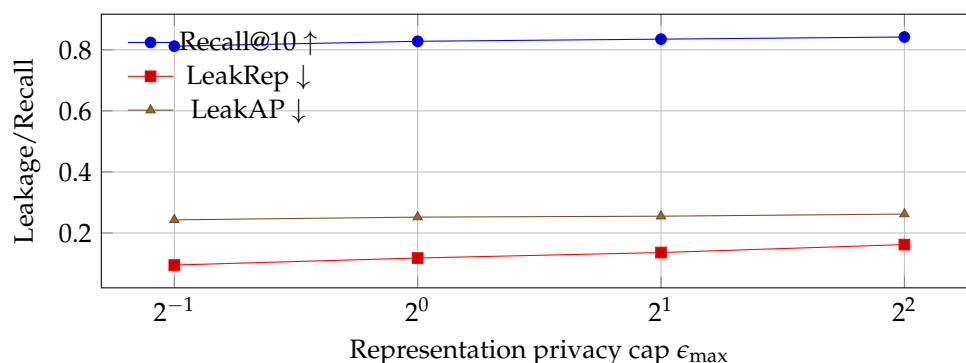


Figure 10. Privacy–utility trade-offs under a sweep of representation privacy caps.

Figure 11 isolates the effect of routing randomness by sweeping the routing-noise scale σ_r (Equation (21)). Increasing σ_r reduces access-trace leakage (lower LeakAP) while gradually reducing recall as probing becomes less deterministic. A practical tuning strategy is, therefore, to choose the smallest σ_r that meets a target LeakAP threshold on a held-

out audit set, and then verify Recall@10 and latency; in Figure 11, $\sigma_r \approx 0.2$ provides a favorable balance.

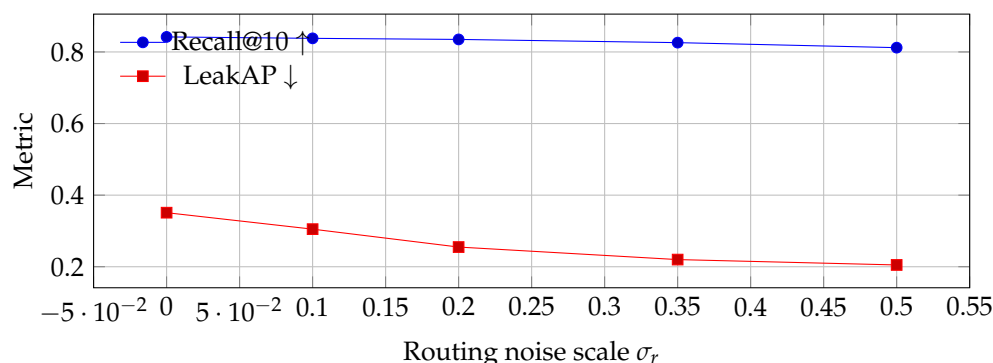


Figure 11. Secure routing sensitivity: increasing σ_r reduces access-trace leakage (LeakAP) with a controlled recall impact.

5.6. Limitations

First, for reproducibility we evaluate on public educational corpora with constructed policy/stream generators; testing on real student submissions, grades, or feedback requires institutional approvals and is left to future work. Second, our formal (ϵ, δ) -DP guarantee applies to the stored-vector release channel under the assumed mechanism $\mathcal{M}$ and composition accounting; query-time access traces are mitigated and measured but are not claimed to be DP. Third, LeakRep/LeakAP are attacker-style evaluations but still simplify richer side-channel models (e.g., very fine-grained timing or hardware channels), which are out of scope. Finally, while we discuss secure search and federated/TEE vector search systems, we do not provide apples-to-apples end-to-end system comparisons because those systems assume different trust boundaries and typically impose different performance costs than the high-throughput single-operator educational setting targeted here.

6. Conclusions

This paper studied Privacy-preserving intelligent management of educational-corpus vector databases. under the practical realities of retrieval-augmented educational systems: continuous updates, strict multi-role access policies, and cumulative privacy risk from repeated embedding releases and deterministic access traces. We proposed PP-EDUVec, a unified framework that (i) structures indexes by policy signatures for policy-first candidate generation, (ii) reduces access-pattern leakage via secure-aware stochastic routing, and (iii) maintains long-horizon privacy compliance while tracking curriculum drift through the budgeted adaptive lifecycle scheduler BARES. Experiments across representative streaming educational workloads showed that PP-EDUVec consistently improves the privacy–utility trade-off, lowering measured representation and access-pattern leakage proxies while preserving strong top- k retrieval quality and reducing maintenance overhead, making it a practical foundation for compliant, high-performance educational vector databases.

Author Contributions: Conceptualization, S.F., F.L., J.Z. and Z.P.; Methodology, S.F. and F.L.; Formal analysis, J.Z.; Project administration, J.C.; Funding acquisition, J.C. All authors have read and agreed to the published version of the manuscript.

Funding: This research was funded by Engineering Research Center for Intelligent Applications of Big Data in Basic Education Teacher Development, Science and Technology Project of Chongqing Municipal Education Commission (Grant No. KJZD-K202401604), and Natural Science Foundation of Chongqing (Grant No. CSTB2024NSCQ-MSX0812).

Data Availability Statement: The original contributions presented in this study are included in the article. Further inquiries can be directed to the corresponding author(s).

Conflicts of Interest: Author Fen Liu was employed by Chongqing Research Institute of China Coal Technology & Engineering Group. The remaining authors declare that the research was conducted in the absence of any commercial or financial relationships that could be construed as a potential conflict of interest.

References

1. Zeng, S.; Zhang, J.; He, P.; Liu, Y.; Xing, Y.; Xu, H.; Ren, J.; Chang, Y.; Wang, S.; Yin, D.; et al. The Good and The Bad: Exploring Privacy Issues in Retrieval-Augmented Generation (RAG). In Proceedings of the Findings of the Association for Computational Linguistics: ACL 2024, Bangkok, Thailand, 11–16 August 2024; pp. 4505–4524. [\[CrossRef\]](#)
2. Fan, Z.; Zeng, Y.; Zheng, Z.; Tong, Y. FedVSE: A Privacy-Preserving and Efficient Vector Search Engine for Federated Databases. *Proc. VLDB Endow.* **2025**, *18*, 5371–5374. [\[CrossRef\]](#)
3. Jégou, H.; Douze, M.; Schmid, C. Product Quantization for Nearest Neighbor Search. *IEEE Trans. Pattern Anal. Mach. Intell.* **2011**, *33*, 117–128. [\[CrossRef\]](#) [\[PubMed\]](#)
4. Malkov, Y.A.; Yashunin, D.A. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Trans. Pattern Anal. Mach. Intell.* **2020**, *42*, 824–836. [\[CrossRef\]](#) [\[PubMed\]](#)
5. Johnson, J.; Douze, M.; Jégou, H. Billion-Scale Similarity Search with GPUs. *IEEE Trans. Big Data* **2021**, *7*, 535–547. [\[CrossRef\]](#)
6. Subramanya, S.; Devvrit, F.; Simhadri, H.V.; Krishnaswamy, R.; Kadekodi, R.; Lal, P.; Bhattacharya, P.; Srinivas, A. DiskANN: Fast Accurate Billion-Point Nearest Neighbor Search on a Single Node. In Proceedings of the Advances in Neural Information Processing Systems (NeurIPS), Vancouver, BC, Canada, 8–14 December 2019.
7. Guo, R.; Sun, P.; Lindgren, E.; Geng, Q.; Simcha, D.; Chern, F.; Kumar, S. Accelerating Large-Scale Inference with Anisotropic Vector Quantization. In *International Conference on Machine Learning (ICML)*; PMLR: Cambridge MA, USA, 2020; pp. 3887–3896.
8. Pan, J.J.; Wang, J.; Li, G. Survey of Vector Database Management Systems. *VLDB J.* **2024**, *33*, 1591–1615. [\[CrossRef\]](#)
9. Pan, J.J.; Wang, J.; Li, G. Vector Database Management Techniques and Systems. In Proceedings of the 2024 International Conference on Management of Data (SIGMOD/PODS) (Tutorial), Santiago, Chile, 9–15 June 2024. [\[CrossRef\]](#)
10. Taipalus, T. Vector Database Management Systems: Fundamental Concepts, Use-Cases, and Current Challenges. *Cogn. Syst. Res.* **2024**, *85*, 101216. [\[CrossRef\]](#)
11. Peng, Z.; Qiao, M.; Zhou, W.; Li, F.; Deng, D. Dynamic Range-Filtering Approximate Nearest Neighbor Search. *Proc. VLDB Endow.* **2025**, *18*, 3256–3268. [\[CrossRef\]](#)
12. Sun, J.; Li, G.; Pan, J.; Wang, J.; Xie, Y.; Liu, R.; Nie, W. GaussDB-Vector: A Large-Scale Persistent Real-Time Vector Database for LLM Applications. *Proc. VLDB Endow.* **2025**, *18*, 4951–4963. [\[CrossRef\]](#)
13. Karpukhin, V.; Oguz, B.; Min, S.; Lewis, P.; Wu, L.; Edunov, S.; Chen, D.; Yih, W.T. Dense Passage Retrieval for Open-Domain Question Answering. In *2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*; Association for Computational Linguistics: Stroudsburg, PA, USA, 2020; pp. 6769–6781.
14. Guu, K.; Lee, K.; Tung, Z.; Pasupat, P.; Chang, M.W. REALM: Retrieval-Augmented Language Model Pre-Training. In *International Conference on Machine Learning (ICML)*; PMLR: Cambridge MA, USA, 2020; pp. 3929–3938.
15. Lewis, P.; Perez, E.; Piktus, A.; Petroni, F.; Karpukhin, V.; Goyal, N.; Küttler, H.; Lewis, M.; Yih, W.T.; Rocktäschel, T.; et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In Proceedings of the Advances in Neural Information Processing Systems (NeurIPS), Virtually, 6–12 December 2020.
16. Izacard, G.; Grave, E. Leveraging Passage Retrieval with Generative Models for Open Domain Question Answering. In Proceedings of the International Conference on Learning Representations (ICLR), Vienna, Austria, 4 May 2021.
17. Fan, W.; Ding, Y.; Ning, L.; Wang, S.; Li, H.; Yin, D.; Chua, T.S.; Li, Q. A Survey on RAG Meeting LLMs: Towards Retrieval-Augmented Large Language Models. In *30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*; ACM: New York, NY, USA, 2024; pp. 6491–6501. [\[CrossRef\]](#)
18. Dwork, C.; McSherry, F.; Nissim, K.; Smith, A. Calibrating Noise to Sensitivity in Private Data Analysis. In *Theory of Cryptography Conference (TCC)*; Lecture Notes in Computer Science; Springer: Berlin/Heidelberg, Germany, 2006; Volume 3876, pp. 265–284. [\[CrossRef\]](#)
19. Dwork, C.; Roth, A. The Algorithmic Foundations of Differential Privacy. *Found. Trends Theor. Comput. Sci.* **2014**, *9*, 211–487. [\[CrossRef\]](#)
20. Abadi, M.; Chu, A.; Goodfellow, I.; McMahan, H.B.; Mironov, I.; Talwar, K.; Zhang, L. Deep Learning with Differential Privacy. In *2016 ACM SIGSAC Conference on Computer and Communications Security (CCS)*; ACM: New York, NY, USA, 2016; pp. 308–318. [\[CrossRef\]](#)

21. Mironov, I. Rényi Differential Privacy. In *2017 IEEE 30th Computer Security Foundations Symposium (CSF)*; IEEE: New York, NY, USA, 2017; pp. 263–275. [\[CrossRef\]](#)
22. Yu, D.; Naik, S.; Backurs, A.; Gopi, S.; Inan, H.A.; Kamath, G.; Kulkarni, J.; Lee, Y.T.; Manoel, A.; Wutschitz, L.; et al. Differentially Private Fine-Tuning of Language Models. *J. Priv. Confidentiality* **2024**, *14*. [\[CrossRef\]](#)
23. Shokri, R.; Stronati, M.; Song, C.; Shmatikov, V. Membership Inference Attacks against Machine Learning Models. In *2017 IEEE Symposium on Security and Privacy (SP)*; IEEE: New York, NY, USA, 2017; pp. 3–18.
24. Song, C.; Ristenpart, T.; Shmatikov, V. Machine Learning Models that Remember Too Much. In *2017 ACM SIGSAC Conference on Computer and Communications Security (CCS)*; ACM: New York, NY, USA, 2017; pp. 587–601. [\[CrossRef\]](#)
25. Carlini, N.; Tramer, F.; Wallace, E.; Jagielski, M.; Herbert-Voss, A.; Lee, K.; Roberts, A.; Brown, T.; Song, D.; Erlingsson, Ú.; et al. Extracting Training Data from Large Language Models. In *30th USENIX Security Symposium (USENIX Security)*; USENIX Association: Berkeley, CA, USA, 2021; pp. 2633–2650.
26. Ferraiolo, D.F.; Kuhn, D.R. Role-Based Access Controls. In *Proceedings of the 15th NIST-NCSC National Computer Security Conference*, Baltimore, MD, USA, 13–16 October 1992; pp. 554–563.
27. Hu, V.C.; Ferraiolo, D.; Kuhn, D.R.; Schnitzer, A.; Sandlin, K.; Miller, R.; Scarfone, K. *Guide to Attribute Based Access Control (ABAC) Definition and Considerations*; Technical Report Special Publication 800–162; National Institute of Standards and Technology (NIST): Gaithersburg, MD, USA, 2014; Includes Updates as of 2019-08-02. [\[CrossRef\]](#)
28. Hu, V.C. *Overview and Considerations of Access Control Based on Attribute Encryption*; Technical Report NIST IR 8450; National Institute of Standards and Technology (NIST): Gaithersburg, MD, USA, 2023. [\[CrossRef\]](#)
29. Curtmola, R.; Garay, J.; Kamara, S.; Ostrovsky, R. Searchable Symmetric Encryption: Improved Definitions and Efficient Constructions. In *13th ACM Conference on Computer and Communications Security (CCS)*; ACM: New York, NY, USA, 2006; pp. 79–88. [\[CrossRef\]](#)
30. Cash, D.; Jarecki, S.; Jutla, C.; Krawczyk, H.; Roşu, M.; Steiner, M. Highly-Scalable Searchable Symmetric Encryption with Support for Boolean Queries. In *Advances in Cryptology—EUROCRYPT*; Springer: Berlin/Heidelberg, Germany, 2013; pp. 353–373. [\[CrossRef\]](#)
31. Li, F.; Ma, J.; Miao, Y.; Liu, X.; Ning, J.; Deng, R.H. A Survey on Searchable Symmetric Encryption. *Acm Comput. Surv.* **2023**, *56*, 1–42. [\[CrossRef\]](#)
32. Gui, Z.; Paterson, K.G.; Patranabis, S.; Warinschi, B. SWISSE: System-Wide Security for Searchable Symmetric Encryption. *Proc. Priv. Enhancing Technol.* **2024**, *2024*, 549–581. [\[CrossRef\]](#)
33. Wong, W.k.; Cheung, D.; Kao, B.; Mamoulis, N. Secure kNN Computation on Encrypted Databases. In *2009 ACM SIGMOD International Conference on Management of Data (SIGMOD)*; ACM: New York, NY, USA, 2009; pp. 139–152. [\[CrossRef\]](#)
34. Gama, J.; Žliobaitė, I.; Bifet, A.; Pechenizkiy, M.; Bouchachia, A. A Survey on Concept Drift Adaptation. *Acm Comput. Surv.* **2014**, *46*, 1–37. [\[CrossRef\]](#)
35. Hinder, F.; Vaquet, V.; Hammer, B. One or Two Things We Know about Concept Drift—A Survey on Monitoring in Evolving Environments. Part A: Detecting Concept Drift. *Front. Artif. Intell.* **2024**, *7*, 1330257. [\[CrossRef\]](#) [\[PubMed\]](#)
36. O’Callaghan, L.; Mishra, N.; Meyerson, A.; Guha, S.; Motwani, R. Streaming-Data Algorithms for High-Quality Clustering. In *2002 IEEE International Conference on Data Engineering (ICDE)*; IEEE: New York, NY, USA, 2002; pp. 685–694. [\[CrossRef\]](#)
37. Chan, T.H.H.; Shi, E.; Song, D. Private and Continual Release of Statistics. In *Theory of Cryptography Conference (TCC)*; Springer: Berlin/Heidelberg, Germany, 2011; pp. 405–434.
38. Romero, C.; Ventura, S. Educational Data Mining: A Review of the State of the Art. *IEEE Trans. Syst. Man, Cybern. Part C (Appl. Rev.)* **2010**, *40*, 601–618. [\[CrossRef\]](#)
39. Siemens, G.; Baker, R.S.J.d. Learning Analytics and Educational Data Mining: Towards Communication and Collaboration. In *2nd International Conference on Learning Analytics and Knowledge (LAK)*; ACM: New York, NY, USA, 2012; pp. 252–254.
40. Slade, S.; Prinsloo, P. Learning Analytics: Ethical Issues and Dilemmas. *Am. Behav. Sci.* **2013**, *57*, 1510–1529. [\[CrossRef\]](#)
41. Nissenbaum, H. Privacy as Contextual Integrity. *Wash. Law Rev.* **2004**, *79*, 119–158.
42. Jones, K.M.L.; Hinchliffe, L.J. Ethical Issues and Learning Analytics: Are Academic Library Practitioners Prepared? *J. Acad. Librariansh.* **2023**, *49*, 102621. [\[CrossRef\]](#)
43. Soffer, T.; Cohen, A. Privacy versus Pedagogy—Students’ Perceptions of Using Learning Analytics in Higher Education. *Australas. J. Educ. Technol.* **2024**, *40*, 14–30. [\[CrossRef\]](#)

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.